Expert Systems with Applications 41 (2014) 3450–3459

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

On-shelf utility mining with negative item values

Guo-Cheng Lan [a,*], Tzung-Pei Hong [b,c], Jen-Peng Huang [d], Vincent S. Tseng [a]

[a] Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan City 701, Taiwan
[b] Department of Computer Science and Information Engineering, National University of Kaohsiung, Kaohsiung City 811, Taiwan
[c] Department of Computer Science and Engineering, National Sun Yat-Sen University, Kaohsiung City 804, Taiwan
[d] Department of Information Management, Southern Taiwan University of Science and Technology, Tainan City 710, Taiwan

ARTICLE INFO

Keywords:
Data mining
Utility mining
On-shelf utility mining
High on-shelf utility itemset
Negative profit

ABSTRACT

On-shelf utility mining has recently received interest in the data mining field due to its practical considerations. On-shelf utility mining considers not only profits and quantities of items in transactions but also their on-shelf time periods in stores. Profit values of items in traditional on-shelf utility mining are considered as being positive. However, in real-world applications, items may be associated with negative profit values. This paper proposes an efficient three-scan mining approach to efficiently find high on-shelf utility itemsets with negative profit values from temporal databases. In particular, an effective itemset generation method is developed to avoid generating a large number of redundant candidates and to effectively reduce the number of data scans in mining. Experimental results for several synthetic and real datasets show that the proposed approach has good performance in pruning effectiveness and execution efficiency.

1. Introduction

Data mining techniques can extract useful information from databases. Among various techniques in data mining, association-rule mining is important due to its consideration of the co-occurrence relationship of items. That is, association-rule mining techniques can be applied to find items with high frequency in a set of transactions (Agrawal, Imielinksi, & Swami, 1993), and thus have many practical applications, such as analyzing purchasing behaviors in retailing stores, traversal behaviors on websites, and so on. Agrawal et al. proposed the most well-known algorithm for mining association rules from a transaction database, called Apriori (Agrawal & Srikant, 1994). However, when the occurrences of items are considered, it is insufficient to evaluate the significance of items in a database. The main reason is that a transaction in a transaction database usually also includes the quantities bought of items and item prices. The same significance in association-rule mining is assumed for all items in a database and thus the actual significance of an itemset cannot be easily recognized. To address the above problem, Chan et al. proposed utility mining (Chan, Yang, & Shen, 2003), which considers both the profits and quantities of products (items) in a set of transactions to evaluate actual utility values of product combinations (itemsets). In their study, itemsets whose actual utility values are larger than or equal to a predefined minimum utility threshold are output as high-utility itemsets. Several studies have modified utility mining for various practical applications, such as improving its performance and the development of incremental utility mining and stream utility mining. The profits of items in these studies were assumed to be positive values.

Temporal data mining has attracted a lot of attention due to its practicality (Ale & Rossi, 2000; Chang, Chen, & Lee, 2002; Lee, Lin, & Chen, 2001; Li, Ning, Wang, & Jajodia, 2003; Ozden, Ramaswamy, & Silberschatz, 1998; Roddick & Spiliopoilou, 2002). For example, consider the product combination {overcoats, stockings}. This combination may not be frequent throughout the entire database, but may have high frequency in winter. Mining time-related knowledge is thus interesting and useful. Of note, since some products in a store may be put on the shelf and taken off it repeatedly, some biases may exist in the discovered association rules. Thus it is necessary to consider the on-shelf time periods of products.

To address this problem, Lan et al. presented a new issue named on-shelf utility mining (Lan, Hong, & Tseng, 2011) to obtain more accurate utility values of itemsets in temporal databases. In Lan et al.'s study (Lan et al., 2011), on-shelf utility mining considered not only quantities and profits of items in transactions but also the on-shelf time periods of the items. Thus, using on-shelf time periods, the actual utility values of itemsets in a temporal database can be accurately evaluated, and also a two-phase algorithm (named *TP-HOU*) was designed to find high-on-shelf-utility itemsets in temporal databases.

* Corresponding author. Tel.: +886 920 231609.
E-mail addresses: rrfoheiay@gmail.com (G.-C. Lan), tphong@nuk.edu.tw (T.-P. Hong), jehuang@mail.stust.edu.tw (J.-P. Huang), tsengsm@mail.ncku.edu.tw (V.S. Tseng).

0957-4174/$ - see front matter © 2013 Elsevier Ltd. All rights reserved.
http://dx.doi.org/10.1016/j.eswa.2013.10.049

Although the traditional utility mining considered the profits and quantities of items to find useful information with high-profit from transactions databases, the profit values of all items in databases were usually identified as positive values. In real-world applications, the profit values of items in stores may be possibly considered as negative values. For example, when supermarkets want to promote some specific products to attract customers' attentions, the specific products and some free products are usually packaged to sell together. In such scenario, customers can receive not only the specific products but also the free products. With the help of the free products, the supermarkets may earn more profits since customers may buy more relevant products of the free products. However, the profit values of free products in the scenario are usually negative values during supermarket promotion due to the consideration of the costs of the free products. To solve this problem, Chu et al. extended the traditional two-phase utility mining approach to find high-utility itemsets with the consideration of negative item profits from transaction databases (Chu, Tseng, & Liang, 2008). However, since the on-shelf time periods of items were not considered in their study (Chu et al., 2008), accurate utility values of the items were not obtained and thus some itemsets with high utility values may not be found using this approach. To address this, we propose an efficient three-scan mining approach (abbreviated as *TS-HOUN*) for discovering high utility itemsets with negative item values from a transaction database. The major contributions of this work are summarized as follows:

1. This work presents a new issue named on-shelf utility mining with the consideration of negative item values. Different from the existing *TP-HOU* approach in Lan et al. (2011), an efficient three-scan approach for discovering high utility itemsets with negative item values is proposed in this study. To our best knowledge, this is the first work on mining high utility itemsets with considering both of the on-shelf time periods and negative values of items in the field of utility mining.
2. An effective upper-bound model with the consideration of with negative item values is designed in the proposed *TS-HOUN* approach to keep the downward-closure property in mining. In addition, an effective itemset generation method is designed to quickly and directly produce promising itemsets from transactions by using the relationship information of items, and thus unnecessary evaluation in mining can also be avoided. Based on the model and method, the proposed *TS-HOUN* only needs three data scans to finish the mining tasks.
3. In the experimental evaluation, several synthetic and real datasets are used to evaluate the performance of the proposed *TS-HOUN* algorithm with the baseline comparison algorithm. The results show the proposed *TS-HOUN* has better performance in terms of both of pruning effectiveness and execution efficiency with respect to varied parameters.

The rest of this paper is organized as follows. Related works are reviewed in Section 2. The problem to be solved and definitions are stated in Section 3, and the proposed three-scan mining algorithm for finding high-on-shelf-utility itemsets with negative item values is described in Section 4. An example is given to illustrate the execution of the proposed algorithm in Section 5. The experimental results are shown in Section 6, and conclusions and suggestions for future work are given in Section 7.

2. Review of related works

In this section, studies related to temporal association-rule mining, utility mining, and on-shelf utility mining are briefly reviewed.

2.1. Temporal association-rule mining

Traditional association-rule mining (Agrawal & Srikant, 1994; Agrawal et al., 1993) can be used to find the frequency relationships among items in various types of databases. In real-world applications, however, each transaction in a database usually includes a time stamp, such as transactions with occurrence time stamps in retailing business stores. For the analysis of temporal data, temporal data mining was then proposed to find temporal patterns and regularity from a set of data. Forms of temporal patterns include sequential patterns (Pei et al., 2004), periodical association rules (Roddick & Spiliopoilou, 2002), cyclic association rules (Ozden et al., 1998), and calendar association rules (Li et al., 2003).

Ale et al. first proposed a mining approach for finding temporal association rules from a temporal transaction database (Ale & Rossi, 2000). In their study (Agrawal et al., 1993), however, they only considered the transaction periods of products, but not their on-shelf periods. Some products might be frequent if only their transaction periods are considered, but infrequent if their on-shelf periods are considered. Afterward, Chang et al. considered the contiguous time periods of products, which was defined as the contiguous time periods between a product's first and last on-shelf time periods (Chang et al., 2002). That is, the on-shelf periods of a product combination in their study were taken as the common on-shelf periods of all the products in the combination. Based on Chang et al.'s study (Chang et al., 2002), Lee et al. extended the concept of common on-shelf periods to publication databases, such as transaction databases for bookstores or DVD rental stores, to find general temporal association rules (Lee et al., 2001). However, they did not consider that a product might be put on a shelf and taken off it multiple times. Thus, the present study considers the individual on-shelf time periods of products instead of the whole on-shelf period.

2.2. Utility mining

In real-world applications, transactions usually contain the profits and quantities of products. Some high-profit products may rarely occur in a transaction database. Product combinations with high profit but low frequency may not be found using association-rule mining approaches. To address this problem, Chan et al. proposed a new research issue named utility mining (Chan et al., 2003), which considered not only the quantities of items but also their profits in a set of transactions. By using these two kinds of information, the actual utility of an item in a database can be more accurately recognized compared to that obtained using traditional association-rule mining, which only considers the frequencies of items.

However, the downward-closure property in utility mining cannot be maintained due to its utility function. To address this problem, Liu et al. proposed a two-phase mining algorithm for discovering high-utility itemsets from a database by adopting a new downward-closure property (Liu & Qu, 2012) called the transaction-weighted utilization (*TWU*) model. In the *TWU* model, the summation (called transaction utility) of utility values of all items in a transaction is used as the upper-bound value of any sub-itemset in the transaction. The transaction-weighted utility of an itemset is defined as the sum of the transaction utility values of the transactions including the itemset in the database (Liu & Qu, 2012). In addition, Liu et al. proposed a two-phase algorithm to find high-utility itemsets from a database. In the first phase, all high-utility upper-bound itemsets are found from a database by using the *TWU* model (Liu & Qu, 2012). Next in the second phase, an additional data scan is required to find the actual utility values of these high-utility upper-bound itemsets. Itemsets with utility values larger than or equal to a predefined threshold (minimum utility threshold) are output as the high-utility itemsets.

2.3. On-shelf utility mining

Several studies (Hu & Mojsilovic, 2007; Lin, Lan, & Hong, 2012; Li, Yeh, & Chang, 2008; Liu & Qu, 2012; Tseng, Chu, & Liang, 2006; Yao & Hamilton, 2006; Yeh, Chang, & Wang, 2008; Yao, Hamilton, & Butz, 2004) have extended utility mining to various practical problems, such as efficiency improvement of utility mining (Hu & Mojsilovic, 2007; Li et al., 2008; Liu & Qu, 2012; Yao & Hamilton, 2006), incremental process of utility mining (Yeh et al., 2008), and utility mining in stream environments (Tseng et al., 2006). However, these studies (Hu & Mojsilovic, 2007; Lin et al., 2012; Li et al., 2008; Liu & Qu, 2012; Tseng et al., 2006; Yao & Hamilton, 2006; Yeh et al., 2008) did not consider the on-shelf time periods of products in stores. To address this problem, Lan et al. then proposed a new issue named on-shelf utility mining, which considered the on-shelf periods of items (Lan et al., 2011) in addition to the quantities and profits of items. In addition, they proposed a two-phase approach, derived from a traditional two-phase utility mining algorithm, for finding high-on-shelf-utility itemsets from a temporal database (Lan et al., 2011). In the first phase, all high-utility itemsets within a given time period are found and considered as possible high-on-shelf-utility itemsets. Next in the second phase, an additional data scan is done to update the actual on-shelf utility values of the itemsets within their on-shelf periods. Finally, itemsets whose actual on-shelf utility values are larger than or equal to a predefined minimum on-shelf utility threshold are output as high-on-shelf-utility itemsets.

Due to some items with negative profit values during a supermarket promotion, Chu et al. then proposed utility mining with negative item profits (Chu et al., 2008). They also developed a two-phase algorithm, similar to the traditional two-phase utility mining algorithm, for finding high-utility itemsets with negative item profits from a transaction database. Note that the transaction utility of a transaction in Chu et al.'s study (Chu et al., 2008) was the summation of the utility values of only the items with positive item profits in that transaction. This was done to maintain the downward-closure property for mining. Li et al. applied the concept of negative item profits to the problem of utility mining in data streams (Li, Huang, & Lee, 2011), and proposed several algorithms for finding high-utility itemsets with negative item profits (Li et al., 2011). However, some biases may exist when on-shelf periods of items are not considered. To obtain more accurate utility values of itemsets in temporal databases, considering the on-shelf time periods of items in stores is necessary. The present study thus explores the efficient mining of high-on-shelf-utility itemsets with negative item profits in temporal databases.

3. Problem statement and definitions

To clearly explain the problem of on-shelf utility mining with negative item profits, consider twelve transactions, each of which consists of three features, namely transaction identification (*TID*), transaction occurrence time, and items purchased. The occurrence time of each transaction is transformed into the corresponding time period. As an example, the data after the transformation shown in Table 1 are used for mining, where the numbers represent the purchased quantities. The individual profit and on-shelf time period of each item are shown in Tables 2 and 3, respectively.

By using the three tables, a set of terms related to on-shelf utility mining with negative item profits can be defined as follows.

Table 1
Twelve transactions for this example.

Period	*TID*	*A*	*B*	*C*	*D*	*E*	*F*
t_1	$Trans_1$	14	0	0	0	2	0
	$Trans_2$	0	3	2	0	0	1
	$Trans_3$	7	1	0	0	0	2
	$Trans_4$	0	3	2	0	0	0
t_2	$Trans_5$	4	0	0	2	0	0
	$Trans_6$	0	1	1	1	0	0
	$Trans_7$	5	0	0	2	0	0
	$Trans_8$	4	1	0	0	0	0
t_3	$Trans_9$	6	0	2	2	0	0
	$Trans_{10}$	0	0	0	3	2	2
	$Trans_{11}$	0	0	0	4	0	2
	$Trans_{12}$	0	0	1	3	3	0

Table 2
Predefined profit values of the items.

Item	Profit
A	1
B	10
C	−3
D	4
E	3
F	−2

Table 3
On-shelf time periods of the items.

Item	Period		
	t_1	t_2	t_3
A	1	1	1
B	1	1	0
C	1	1	1
D	0	1	1
E	1	0	1
F	1	0	1

Definition 1. $T = \{t_1, t_2, \ldots, t_j, \ldots, t_n\}$ is a set of mutually disjoint time periods, where t_j denotes the jth time period in the whole set of periods, T. For example, in Table 1, T includes three time periods, namely t_1, t_2, and t_3.

Definition 2. $I = \{i_1, i_2, \ldots, i_m\}$ is a set of items, which may appear in transactions.

Definition 3. An itemset X is a set of items, $X \subseteq I$. If $|X| = k$, the itemset X is called a k-itemset. For example, a 2-itemset $\{BC\}$ contains two items, B and C.

Definition 4. A temporal transaction (*Trans*) is composed of a set of purchased items with their quantities and time period t in T.

Definition 5. A temporal transaction database D is composed of transaction sets of several time periods. That is, $D = \{Trans_{1.1}, Trans_{1.2}, \ldots, Trans_{j.y}, \ldots, Trans_{n.m}\}$, where $Trans_{j.y}$ is the yth transaction in the jth time period. Moreover, all transactions in the jth time period t_j can be identified as D_j. For example, in Table 1, D_1 includes $Trans_1$, $Trans_2$, $Trans_3$, and $Trans_4$ in the first time period t_1.

Definition 6. The local temporal transaction utility $q(i, Trans_{j.y})$ of an item i in a transaction $Trans_{j.y}$ is the quantity of i in $Trans_{j.y}$. For example, $q(A, Trans_{2.8}) = 1$.

Definition 7. The external utility $s(i)$ of an item i is the corresponding profit of each item in the utility table. Note that the profit of an item can be considered as positive or negative to reflect its importance. For example, in Table 2, $s(C) = -3$.

Definition 8. The utility $u(i, Trans_{j,y})$ of an item i in a temporal transaction $Trans_{j,y}$ is the external utility $s(i)$ of i in the utility table multiplied by the local temporal transaction utility $q(i,Trans_{j,y})$ of i in $Trans_{j,y}$. That is:

$$u(i,\ Trans_{j,y}) = s(i) * q(i,\ Trans_{j,y}).$$

For example, in Table 1, $u(A,Trans_{1,3}) = 1*7 = 7$.

Definition 9. The transaction utility $tu(Trans_{j,y})$ of the yth temporal transaction $Trans_{j,y}$ is the sum of the utility values of all the items contained in $Trans_{j,y}$. That is:

$$tu(Trans_{j,y}) = \sum_{i \in Trans_{j,y}} u(i,\ Trans_{j,y}).$$

For example, $tu(Trans_{1,4}) = u(B, Trans_{1,4}) = (10 * 3) = 30$.

Definition 10. The utility $u(X,Trans_{j,y})$ of an itemset X in a temporal transaction $Trans_{j,y}$ is the sum of the utility values of all items in X in $Trans_{j,y}$. That is:

$$u(X,\ Trans_{j,y}) = \sum_{i \in X} u(i, Trans_{j,y}).$$

For example, in Table 1, $u(\{BC\},Trans_{1,2}) = 10 * 3 + -3 * 2 = 24$.

Definition 11. The periodical utility $pu(X,t_j)$ of an itemset X is the sum of the utility values of X in all transactions including X within the jth period t_j. That is:

$$pu(X,\ t_j) = \sum_{Trans_{j,y} \subseteq D_j \wedge X \subseteq Trans_{j,y}} u(X,\ Trans_{j,y}).$$

For example, in Table 1, $pu(\{BC\},t_1) = u(\{BC\},Trans_{1,2}) + u(\{BC\},Trans_{1,4}) = (10 * 3 + -3 * 2) + (10 * 3 + -3 * 2) = 48$.

Definition 12. The periodical total transaction utility $pttu(t_j)$ is the sum of the transaction utilities of all transactions within the jth time period t_j. That is:

$$pttu(t_j) = \sum_{Trans_{j,y} \subseteq D_j} tu(Trans_{j,y}).$$

For example, $pttu(t_1) = tu(Trans_{1,1}) + tu(Trans_{1,2}) + tu(Trans_{1,3}) + tu(Trans_{1,4}) = 20 + 30 + 17 + 30 = 97$.

Definition 13. The periodical utility ratio $pur(X,t_j)$ of an itemset X is the periodical utility $pu(X,t_j)$ of X within the jth period t_j over the periodical total utility $pttu(t_j)$ of the time period t_j. That is:

$$pur(X, t_j) = pu(X, t_j)/pttu(t_j).$$

For example, $pur(\{BC\},t_1) = pu(\{BC\},t_1)/pttu(t_1) = 48/97 = 49.48\%$.

Definition 14. Let λ be a pre-defined minimum on-shelf utility threshold. An itemset X is called a high-periodical-utility itemset (*HPU*) if $pur(X) \geqq \lambda$. For example, if $\lambda = 30\%$, then $\{BC\}$ is a high-periodical-utility itemset.

Here, discovery of *HPUs* in a time period is the same as that of high-utility itemsets in traditional utility mining (Liu & Qu, 2012) in a database. The above definitions are further extended to on-shelf utility mining.

Definition 15. The on-shelf time periods of an item i, $op(i)$, are for sale on the shelf within a set of time periods. For example, in Table 3, the on-shelf time periods of item B, $op(B)$, are t_1 and t_2.

Definition 16. The common on-shelf time periodsof all items in an itemset X, $op(X)$, are for sale on the shelf within a set of time periods. For example, in Table 3, the common on-shelf time period of the itemset $\{BF\}$, $op(\{BF\})$, is t_2.

Definition 17. The on-shelf utility of an item i, $ou(i)$, is the sum of utilities of i within the union of on-shelf time periods of i. For example, in Table 1, $ou(\{B\}) = pu(\{B\},t_1) + pu(\{B\}, t_2) = (10 * 3 + 10 * 1 + 10 * 3) + (10 * 1 + 10 * 1) = 70 + 20 = 90$.

Definition 18. The on-shelf utility of an itemset X, $opu(X)$, is the sum of utilities of X within the union of on-shelf periods of X. For example, in Table 1, $ou(\{BC\}) = pu(\{BC\},t_1) + pu(\{BC\}\_,t_2) = (24 + 24) + (7) = 55$.

Definition 19. The on-shelf utility ratio of an itemset X, $our(X)$, is the sum of utilities of X within all on-shelf time periods of X over the sum of all the transaction utilities within the union of the on-shelf time periods of X. For example, in the three tables, $our(\{BC\}) = ou(\{BC\})/(pttu(t_1) + pttu(t_2)) = (55)/(97 + 53) = 36.67\%$.

Definition 20. An itemset X is called a high-on-shelf-utility itemset (*HOU*) if $our(X) \geqq \lambda$. For example, if λ is set at 30%, $\{BC\}$ is an *HOU*.

The downward-closure property in on-shelf utility mining with negative item profits cannot be maintained. For example, if $\lambda = 30\%$, then $our(\{A\}) = 18.26\% < \lambda$; $\{A\}$ is thus a low-on-shelf-utility item in Table 1. However, its superset $\{AD\}$ is a high-on-shelf-utility itemset. To handle this, the concept of the *TWU* model (Liu & Qu, 2012) is extended to design an effective filtering condition in on-shelf utility mining with negative item profits. The relevant terms are defined as follows.

Definition 21. The periodical utility upper-bound $puu(X,t)$ of an itemset X within one time period t is the sum of the transaction utility values of all transactions including X within t. That is:

$$puur(X,\ t) = \sum_{Trans_{j,y} \subseteq D_j \wedge X \subseteq Trans_{j,y}} tu(Trans_{j,y}).$$

For example, $puu(\{B\},\ t_1) = (30 + 10 + 30) = 70$.

Definition 22. The periodical utility upper-bound ratio $puur(X,t)$ of an itemset X is the sum of the transaction utility values of all transactions including X over the sum of all the transaction utility values within one time period t. That is:

$$puur(X, t) = puu(X, t)/pttu(t).$$

For example, $puur(\{B\},\ t_1) = 70/97 = 72.16\%$.

Definition 23. An itemset X is called a high-periodical-utility upper-bound itemset (*HPUU*) within the jth time period, if $puur(X,t_j) \geqq \lambda$. For example, if $\lambda = 30\%$, then the itemset $\{B\}$ is an *HPUU* since $puur(\{B\},\ t_1) = 72.16\% \geqq \lambda$.

Problem statement: Based on the above definitions, the problem to be solved is to find the itemsets whose actual on-shelf utility values within the union of their on-shelf time periods are larger than or equal to a predefined minimum on-shelf utility threshold. Note that the profits of the items can be either positive or negative values. To solve this problem, this work presents a three-scan mining algorithm for finding *HOUs* with negative item values in temporal databases.

4. Proposed algorithm

In this paper, an effective itemset-generation method is proposed to speed up the execution efficiency of the proposed *TS-HOUN* approach for mining. The itemset-generation method is described below.

4.1. Itemset-generation method

This work designs an itemset-generation approach and a filtration mechanism to effectively reduce the number of data scans and unpromising candidates in mining. The high-periodical-utility upper-bound 2-itemsets within each time period are used to prune unpromising candidate itemsets. An example is given below to illustrate the proposed method.

Consider a transaction *T* in a certain time period that includes four items, 3*A*, 2*B*, 25*C*, and 3*D*, where the numbers represent the quantities of the items. Their profit values are 3, 10, 1, and 6, respectively. Suppose that {*AB*}, {*BC*}, and {*CD*} are three high-periodical-utility upper-bound 2-itemsets within this time period that have been found. Fig. 1 shows the process of generating itemsets using the proposed method and filtration mechanism.

In Fig. 1, the proposed method first fetches the first item *A* in *T* and places it in the first row of a two-dimensional vector array. The method then fetches the second item *B* in *T* and places it in the second row of the vector array. Since there is only the item *A* in front of the fetched item *B*, the method checks whether items *A* and *B* have a high-periodical-utility upper-bound relationship. In this example, {*AB*} is a high-periodical-utility upper-bound 2-itemset. Thus {*AB*} is generated and placed after {*A*} in the first row because the first item in {*AB*} is *A*. Next, the method fetches the third item *C* and performs the same process. It puts {*C*} in the third row of the vector array. {*AC*} is formed in the first row and it is checked whether {*AC*} is a high-periodical-utility upper-bound 2-itemset. {*AC*} is not, so no combination of a subset in the first row with {*C*} is necessary. The method then forms {*BC*} from the second row and determines whether {*BC*} is a high-periodical-utility upper-bound 2-itemset. {*BC*} is thus placed after {*B*} in the second row. Two new subsets, {*BC*} and {*C*}, are generated for the third item. Similarly, the fourth item *D* is fetched and the above process is repeated. The two new subsets {*CD*} and {*D*} are derived and put into the vector array.

As shown in this example, a transaction is only scanned once to generate possible itemsets using the proposed itemset-generation method. This method thus effectively reduces the number of required data scans for mining.

4.2. Proposed TS-HOUN algorithm

In our previous study (Lan et al., 2011), the main principle of the *TP-HOU* approach was purely extended from the two-phase utility mining approach in Liu et al.'s study (Liu, Liao, & Choudhary, 2005). That is, the *TP-HOU* approach (Lan et al., 2011) only adopted the level-wise technique to handle the problem of on-shelf utility mining, it needed multiple data scans to finish the mining task. However, different from the *TP-HOU* approach (Li et al., 2011), this work presents an effective *TS-HOUN* approach that needs only three data scans to finish the problem of on-shelf utility mining with negative item values, and also the original upper-bound model in the *TP-HOU* approach (Li et al., 2011) cannot be directly used to cope with the problem of this work. Hence, a proper upper-bound model with the consideration of negative item values is also developed to keep the downward-closure property in mining (As mentioned in the definitions 21 to 23). The proposed *TS-HOUN* approach for mining high on-shelf-utility itemsets with negative item values is described below.

Input: A temporal transaction database *D* with *n* transactions, each of which consists of a transaction identification, transaction occurrence time, and items purchased, *m* items in *D*, each with a positive or a negative profit value, a table of on-shelf items with *k* desired time periods, and a minimum on-shelf utility threshold λ.

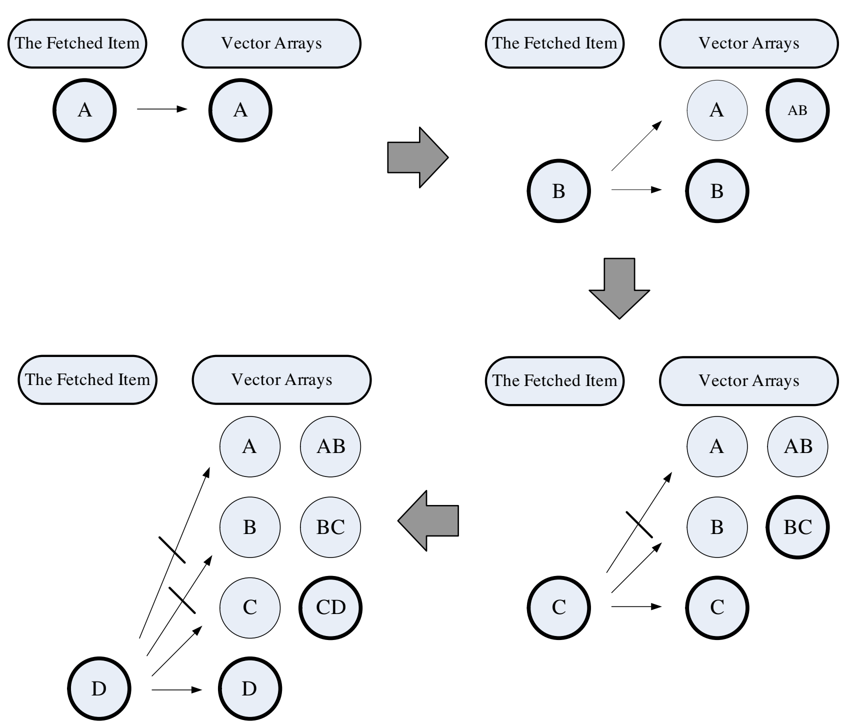

Fig. 1. Process of generating itemsets using filtration mechanism.

Output: A final set of high-on-shelf-utility itemsets with negative values, *HOUN*.
Scan 1:

- Step 1: Transform the occurring time of each transaction into the corresponding time period.
- Step 2: Initialize a periodical total transaction utility (*PTTU*) table as a zero table, in which the row number is the time period number and each entry in the *PTTU* table is set as 0.
- Step 3: For each *y*th transaction $Trans_{jy}$ in each period t_j, do the following substeps.
  - (a) Calculate the utility value $u(i_z, Trans_{jy})$ of each *z*th item i_{jyz} in $Trans_{jy}$.
  - (b) Calculate the sum of utility values of all items with positive item profits in $Trans_{jy}$ to obtain the transaction utility $tu(Trans_{jy})$ of $Trans_{jy}$.
- Step 4: Calculate the sum of transaction utility values of all transactions in each period t_j to obtain the periodical total transaction utility $pttu_j$.
- Step 5: Find in the *PTTU* table the entry which has the same time period as each t_j, and add $pttu_j$ to the entry.
- Step 6: Find the 2-itemsets with high periodical utility upper-bound values in each time period t_j. Denote the set of high-periodical-utility upper-bound 2-itemsets in each t_j as $HPUU_{j2}$ and the union of all $HPUU_{j2}$'s for all time periods as $HPUU_2$. (As mentioned in Section 4.1)

  Scan 2:
- Step 7: Find the itemsets with high periodical utility values in each time periodt_j. Denote the set of high-periodical-utility itemsets in each t_j as HPU_j and the union of all HPU_j's for all time periods as *HPU*.
- Step 8: Initially set *HOUN* as empty.
- Step 9: For each itemset *X* in *HPU,* find its actual on-shelf utility by the following substeps.
  - (a) Use the AND operation to obtain the set of common on-shelf periods (COS_X) of all the items in the itemset *X* from the *OS* table.
  - (b) Sum the actual utilities $u(X)^{appearing}$ of *X* appearing in *HPU* as follows:

$$u(X)^{appearing} = \sum_{X \in HPU \wedge t_j \in COS_X} u_j(X),$$

where $u_j(X)$ is the actual utility of itemset *X* in the time period t_j.

  - (c) If the actual utility value of *X* in each of its on-shelf time periods is known, and $u(X)^{actual}/pttu(X) \geqq \lambda$, then *X* is a high-on-shelf-utility itemset; set $HOUN = HOUN \cup X$ and $HPU = HPU - X$; otherwise, omit itemset *X*.

Scan 3:

- Step 10: For each itemset *X* in *HPU*, do the following substeps.
  - (a) Scan the database to find its actual periodical utility value $u(X)_j^{scan}$ in time period $t_j \in COS_x$ if *X* does not appear in that time period.
  - (b) Calculate the actual on-shelf utility $u(X)^{actual}$ of each itemset *X* in *HPU* as:

$$u(X)^{actual} = u(X)^{appearing} + \sum_{X \in HUI \wedge X \notin HUI_j \wedge t_j \in COS_X} u(X)_j^{scan}.$$

  - (c) If $u(X)^{actual}/pttu(X) \geqq \lambda$, then *X* is a high-on-shelf-utility itemset; set $HOUN = HOUN \cup X$ and $HPU = HPU - X$; otherwise, set $HPU = HPU - X$.
- Step 11: Output the final set of high-on-shelf-utility itemsets with negative item values in *HOUN*.

5. Example for TS-HOUN

In this section, a simple example is given to demonstrate how the proposed *TS-HOUN* algorithm can easily be used to find high-on-shelf-utility itemsets with negative item values from a temporal transaction database *D*. The temporal transaction database, utility table, and on-shelf information table shown in Tables 1–3, respectively, are used in the example. The predefined minimum on-shelf utility threshold λ is set at 30%. The proposed *TS-HOUN* algorithm proceeds as follows.

- STEP 1: The occurrence time of each transaction is first transformed into the corresponding time period (see Table 1).
- STEP 2: The *PTTU* table is initialized as the zero table. In this example, since there are three time periods, only three rows are generated in the *PTTU* table.
- STEP 3: For each transaction in the dataset D_j within each period t_j, the utility value of each item in the transaction and then the transaction utility of the transaction are calculated. Take the third transaction $Trans_3$ in t_1 in Table 1 as an example. The transaction includes the three items, *A*, *B*, and *F*; their quantities in $Trans_3$ are 7, 1, and 2, respectively, and their profits in Table 2 are 1, 10, and −2, respectively. The utility values of the three items in $Trans_3$ can be calculated as $7 * 1(=7)$, $1 * 10(=10)$, and $2* - 2(= -4)$, respectively. The transaction utility of the transaction can be calculated as 7 + 10(=17). All the other transactions in *D* can be similarly processed. The results for the transaction utility values of all transactions are showed in Table 4.
- STEP 4: The periodical total transaction utility (*pttu*) values of the three periods are 97, 53, and 69, respectively.
- STEP 5: The periodical total transaction utility in each t_j is added to the corresponding entry ($pttu_j$) in the *PTTU* table, as shown in Table 5.
- STEP 6: High-periodical-utility upper-bound 2-itemsets within each time period are found. Take the first time period t_1 as an example. The first transaction $Trans_1$ in t_1 includes the two items, *A* and *E*. The transaction utility of $Trans_1$ in Table 4 is 20. Only the possible 2-itemset {*AE*} can be generated from $Trans_1$. {*AE*} and the transaction utility (= 20) of $Trans_1$ are put in the corresponding fields of the $TI_{2,t1}$ table of the first time period t_1.

Next, the periodical utility upper-bound ratio of each 2-itemset can be obtained in Table 6, and then the high-periodical-utility upper-bound 2-itemsets in the first time period, t_1, are found from Table 6. Take the candidate 2-itemset {*AE*} in Table 6 as an example. The periodical utility upper-bound of {*AE*} and the periodical total transaction utility of t_1 are 20 and 97, respectively. Then, the periodical utility upper-bound ratio of {*AE*} can be calculated as 20/97 = 20.62%. The other six 2-itmesets in t_1 can be similarly processed. The periodical utility upper-bound ratio (*puubr*) values of the seven 2-itemsets are showed in Table 7.

In Table 7, the periodical utility upper-bound ratio values of the three 2-itemsets {*BC*}, {*BF*}, and {*CF*} are larger than or equal to the minimum on-shelf utility threshold (= 30%), so these itemsets are put in the set of high-periodical-utility upper-bound 2-itemsets $HPUUB_2$ in t_1. The above process is repeated for the other two time periods, t_2 and t_3. All high-periodical-utility upper-bound 2-itemsets in each time period are showed in Table 8.

- STEP 7: All high-periodical-utility itemsets in each t_j are found from the set of high-periodical-utility upper-bound 2-itemsets. Take the first time period t_1 as an example. The process of generating promising candidates for each transaction is the same as that mentioned in Section 4.2. Based on the itemset-generation method, only two promising itemsets, {*A*} and {*E*}, can be generated from

Table 4
Transaction utilities of all transactions in example.

Period	TID	A	B	C	D	E	F	$tu(Trans_{jy})$
t_1	$Trans_1$	14	0	0	0	2	0	20
	$Trans_2$	0	3	2	0	0	1	30
	$Trans_3$	7	1	0	0	0	2	17
	$Trans_4$	0	3	2	0	0	0	30
t_2	$Trans_5$	4	0	0	2	0	0	12
	$Trans_6$	0	1	1	1	0	0	14
	$Trans_7$	5	0	0	2	0	0	13
	$Trans_8$	4	1	0	0	0	0	14
t_3	$Trans_9$	6	0	2	2	0	0	14
	$Trans_{10}$	0	0	0	3	2	2	18
	$Trans_{11}$	0	0	0	4	0	2	16
	$Trans_{12}$	0	0	1	3	3	0	21

Table 5
PTTU table for example.

Period	*pttu*
t_1	97
t_2	53
t_3	69

Table 6
Set of candidate 2-itemsets in $TI_{2,t1}$ table.

2-Itemset	*puub*	2-Itemset	*puub*
{*AE*}	20	{*AB*}	17
{*BC*}	60	{*AF*}	17
{*BF*}	30	{*BF*}	17
{*CF*}	30		

Table 7
Set of candidate 2-itemsets in $TI_{2,t1}$ table.

2-Itemset	*puubr* (%)	2-Itemset	*puubr* (%)
{*AE*}	20.62	{*AB*}	17.53
{*BC*}	61.86	{*AF*}	17.53
{*BF*}	30.93	{*BF*}	17.53
{*CF*}	30.93		

the first transaction {14*A*, 2*E*} in t_1 by using the set of $HPUUB_2$ of t_1. The same process is repeated for the other three transactions in t_1. The promising itemsets with periodical utilities in t_1 are showed in Table 9.

Next, the periodical utility ratio of each itemset in Table 9 is found and it is determined whether the itemset is a high-periodical-utility itemset. In this example, the periodical utility upper-bound ratio values of the three 2-itemsets {*B*}, {*BC*}, and {*BF*} are larger than or equal to the minimum on-shelf utility threshold (= 30%), so they are put in the set of high-periodical-utility itemsets *HPU* in t_1. The same process is repeated for the other two time periods, t_2 and t_3. The high-periodical-utility itemsets in each time period are showed in Table 10.

STEP 8: The set of *HOUN*, used to maintain the high-on-shelf-utility itemsets and their on-shelf utility ratio values, is initialized as empty.

STEP 9: The utility of each itemset in Table 10 is found. Take the 1-itemset {*B*} in Table 1 as an example. According to Table 3, the on-shelf time periods of {*B*} include all the time periods, t_1, t_2, and t_3. The periodical utilities of {*B*} in t_1 and t_2 in Table 10 are known, with values of 70 and 20, respectively. The utility of {*B*} can be calculated as 70 + 20 = 90. However, since the periodical utility of {*B*} in t_3 is unknown, the actual on-shelf utility of {*B*} is unknown. Thus, {*B*} is omitted. The same process is repeated for the other four itemsets in Table 10. The four itemsets {*B*}, {*D*}, {*BF*}, and {*DE*} are put into *HOUN* because their on-shelf utility values are known within their on-shelf time periods and they are larger than or equal to the minimum on-shelf utility threshold (= 30%). Hence, the four itemsets are removed from Table 10, with only itemset {*BC*} remaining.

STEP 10: For each itemset still in Table 10, its unknown periodical utility in each of its on-shelf time periods is found if it is not a high-periodical-utility itemset in that time period. Take the itemset {*BC*} in Table 10 as an example. Since the periodical utility of {*BC*} in t_2 is unknown, the transactions in t_2 are scanned to find the periodical utility of {*BC*}. After the data scan, the periodical utility of {*BC*} is found to be 17. Then, the on-shelf utility of {*BC*} can be calculated as (48 + 17)/(97 + 65), which is 40.12%. Since the on-shelf utility ratio of the itemset {*BC*} is larger than or equal to the minimum on-shelf utility threshold, {*BC*} is a high-on-shelf-utility itemset and put into *HOUN*. {*BC*} is then removed from Table 10. In this example, since there are no more itemsets in Table 10, the process of finding high-on-shelf-utility itemsets can be terminated. The found high-on-shelf-utility itemsets are showed in Table 11.

STEP 11: The five high-on-shelf-utility itemsets in Table 11 are output as decision makers' auxiliary information.

As noted in this example, the proposed *TS-HOUN* algorithm only needs three data scans to finish the mining task. *TS-HOUN* can effectively reduce the number of data scans with the help of the proposed itemset-generation method. Its execution efficiency is thus higher than those of traditional Apriori-based algorithms.

Table 8
Set of candidate 2-itemsets in $TI_{2,t1}$ table.

Period	$HPUUB_2$	*puubr* (%)
t_1	{*BC*}	61.86
	{*BF*}	30.93
	{*CF*}	30.93
t_2	None	None
t_3	{*CD*}	50.72
	{*DE*}	56.52
	{*DF*}	49.28
	{*CE*}	30.43

Table 9
Set of promising itemsets with periodical utilities in t_1.

Itemset	Periodical utility	Itemset	Periodical utility
{*A*}	21	{*F*}	−6
{*B*}	70	{*BC*}	48
{*C*}	−12	{*BF*}	34
{*E*}	6	{*CF*}	−8

Table 10
Set of candidate 2-itemsets in $TI_{2,t1}$ table.

Period	*HPU*	*pu*
t_1	{*B*}	70
	{*BC*}	48
	{*BF*}	34
t_2	{*B*}	20
	{*D*}	20
t_3	{*D*}	48
	{*DE*}	39

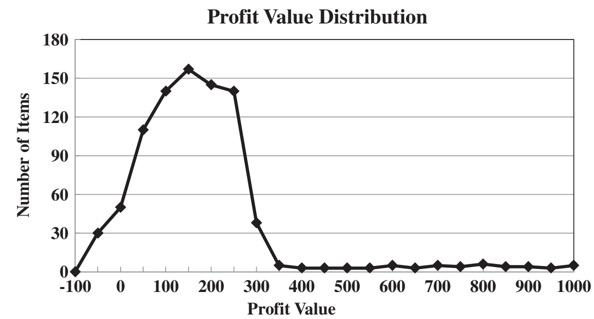

Fig. 2. Profit-value distribution based on the utility table.

6. Experimental evaluation

A series of experiments was conducted to compare the performance of the proposed *TS-HOUN* algorithm in terms of pruning and efficiency. To show practical performance of the *TS-HOUN* approach, a proper mining approach that was extended from Lan et al.'s two-phase on-shelf utility mining approach (Lan et al., 2011) was designed as a baseline approach (named *TP-HOUN*). The experiments were implemented in J2SDK 1.6.0 and executed on a PC with a 3.0-GHz CPU and 1 GB of memory.

6.1. Experimental datasets

In the experiments, a public IBM data generator (IBM) was used to produce the datasets (Hu & Mojsilovic, 2007). Since the purpose was to find high-on-shelf-utility itemsets, we developed a simulation model similar to that used in Chu et al.'s study (Chu et al., 2008) to generate the quantity values of the items in the transactions. Each quantity ranged from 1 to 5, as described in Chu et al. (2008). An integer value from 1 to the total number of given time periods was randomly assigned to each transaction in the generated datasets. Moreover, for each dataset generated, a corresponding utility table was produced in which a profit value in the range of -100 to 1000 was randomly assigned to an item. Fig. 2 shows the profit-value distribution of all items generated by the model in the utility table.

A real dataset, BMS-POS (Frequent Itemsets Mining Dataset Repository), was also used in the experiments. The dataset, used in the KDDCUP 2000 competition, contains several years of point-of-sale data from a large electronics retailer, making it very suitable as a test dataset. There are 515,597 transactions in this dataset and 1,657 different items. The quantity values and a utility table with negative item values generated by our simulation model were appended to the items in both the transactions and the real dataset.

6.2. Evaluation of differences of two utility itemsets with negative values

The experiments were first made on the synthetic T10I4N4KD200K dataset to evaluate the difference ratio of high-utility itemsets with negative item values and high-on-shelf-utility itemsets with negative item values for time periods P5, P25, and P50. Here a measurement was given to evaluate the difference rate of the proposed high on-shelf utility itemsets with negative item values when compared to the traditional high utility itemsets with negative item values under various parameter settings. The measure was defined as follows:

$$Change\ Rate = \frac{|\ HUN - HOUN\ |}{|\ HOUN\ |},$$

where $|HUN|$ and $|HOUN|$ represent the numbers of high-utility itemsets with negative item values and high-on-shelf-utility itemsets with negative item values, respectively. Figs. 3 and 4 show the differences between these two utility itemsets with negative item values for the T10I4N4KD200K dataset for thresholds in the range of 1.00% to 0.20% and for the T10I4N4K dataset with sizes in the range of 100 k to 500 k and a threshold set at 0.20%, respectively.

From these figures, it could be observed that the difference rates of high utility itemsets with negative values and high on-shelf utility itemsets with negative values were very obvious under different time periods. At least 30% average difference rates of the two kinds of itemsets were achieved. The result meant that considering on-shelf time periods of items allows a higher number of high-utility itemsets with centralized utility values and negative item values to be found. In the experiments, the time period values were randomly assigned for transactions; there was no fixed trend regarding the mined numbers of itemsets.

6.3. Efficiency evaluation

Experiments were conducted to evaluate the efficiency of the two algorithms, *TS-HOUN* and *TP-HOUN*, for time periods P5, P25, and P50. Figs. 5 and 6 show the efficiency for the T10I4N4KD200K dataset for thresholds in the range of 1.00% to 0.20% and for the T10I4N4K dataset with sizes in the range of 100 k to 500 k and a threshold set at 0.20%, respectively.

The figures show that the execution efficiency of the proposed algorithm *TS-HOUN* is higher than that of the conventional algorithm *TP-HOUN*. Since the conventional algorithm *TP-HOUN* adopts the level-wise technique to handle the problem of on-shelf utility mining with negative item profits, *TS-HOUN* outperformed *TP-HOUN* in finding high-on-shelf-utility itemsets.

6.4. Evaluation using a real dataset

The real dataset *BMS-POS* (Frequent Itemsets Mining Dataset Repository) was used to evaluate the performance of *TS-HOUN* and *TP-HOUN* for various thresholds. Figs. 7 and 8 show difference rates of the two utility itemsets with negative item values and execution time required by the two algorithms for thresholds in the range of 0.20% to 1.00%, respectively.

Table 11
Final set of *HOUN* in this example.

Itemset	On-shelf utility ratio (%)	Periodical utility
{*B*}	55.56	$[70_{t1}, 20_{t2}]$
{*D*}	50.75	$[20_{t2}, 48_{t3}]$
{*BC*}	40.12	$[48_{t1}, 17_{t2}]$
{*BF*}	35.05	$[34_{t1}]$
{*DE*}	40.21	$[39_{t3}]$

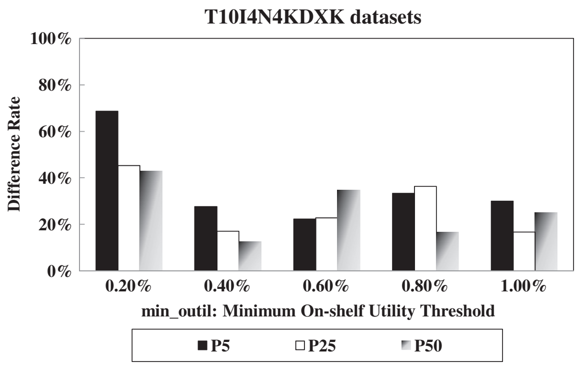

Fig. 3. Difference rates of the two kinds of itemsets for various thresholds.

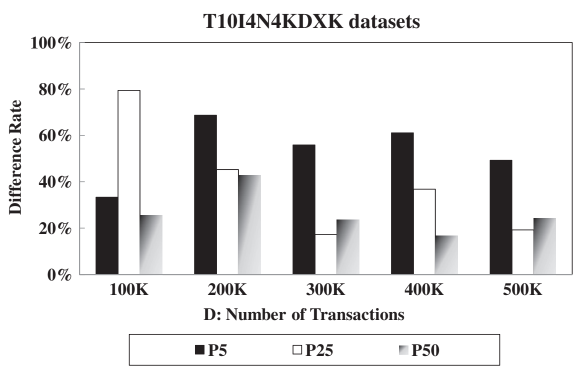

Fig. 4. Difference rates of the two kinds of itemsets for various data sizes.

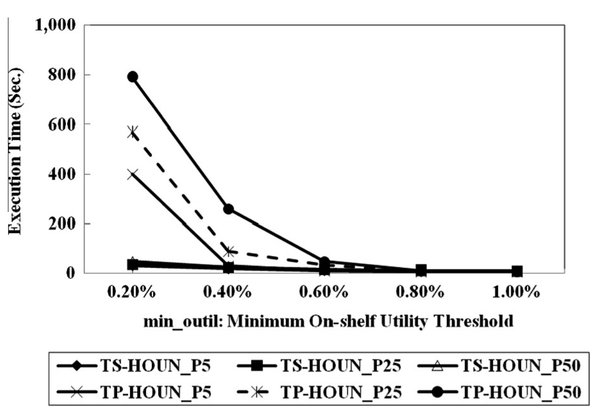

Fig. 5. Execution efficiency of the two algorithms for various thresholds.

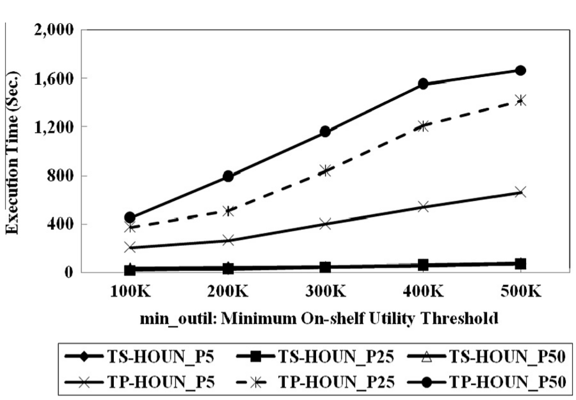

Fig. 6. Execution efficiency of the two algorithms for various data sizes.

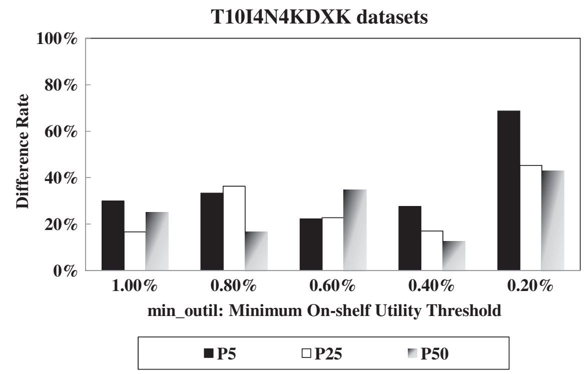

Fig. 7. Difference rates of the two kinds of itemsets for various thresholds.

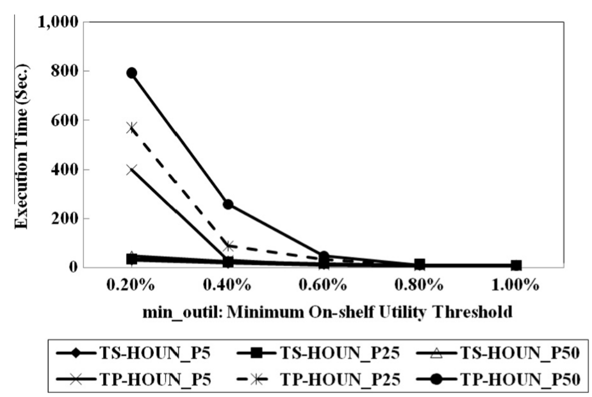

Fig. 8. Execution efficiency of the two algorithms for various thresholds.

The figures show that the difference rates of the two utility itemsets within the three kinds of time periods was still obvious under different thresholds. The proposed *TS-HOUN* algorithm outperformed the baseline *TP-HOUN* algorithm for the real dataset *BMS-POS* with regard to execution efficiency.

7. Conclusion

This work presents a new research issue named on-shelf utility itemset mining with the consideration of negative values of items. In addition, an itemset-generation method that directly produces promising itemsets from transactions in mining is developed to effectively reduce the number of data scans and to speed up the execution efficiency in mining. With the help of the method, the effective mining approach (*TS-HOUN*) only needs three data scans to cope with this problem. At last, the experimental results on synthetic and real datasets show that the effectiveness of the proposed on-shelf utility mining with negative item values, and the results also show the proposed *TS-HOUN* approach has good performance in execution efficiency when compared to the compared approach, *TP-HOUN*.

In the future, we aim to apply the proposed mining approach to other practical applications, such as data streams. In addition, how to further improve the execution efficiency for mining high-on-shelf utility itemsets with negative item values is another interesting topic, such as the development of tree-based mining approaches. Besides, we will also attempt to address the maintenance problem of on-shelf utility mining with negative item values when transactions are inserted, deleted or modified.

References

Agrawal, R., & Srikant, R. (1994). Fast algorithm for mining association rules. In *The international conference on very large data bases* (pp. 487–499).

Agrawal, R., Imielinksi, T., & Swami, A. (1993). Mining association rules between sets of items in large database. In *The ACM SIGMOD international conference on management of data* (pp.207–216).

Ale, J. M., & Rossi, G. H., (2000). An approach to discovering temporal association rules. In *The 2000 ACM symposium on applied computing* (pp. 294–300).

Chan, R., Yang, Q., & Shen, Y. (2003). Mining high utility itemsets. In *The third IEEE international conference on data mining* (pp. 19–26).

Chang, C. Y., Chen, M. S., & Lee, C. H. (2002). Mining general temporal association rules for items with different exhibition periods. In *The third IEEE international conference on data mining* (pp. 59–66).

Chu, C. J., Tseng, Vincent S., & Liang, T. (2008). An efficient algorithm for mining high utility itemsets with negative item values in large databases. *Applied Mathematics and Computation, 215*(2), 767–778.

Frequent Itemsets Mining Dataset Repository. Available at (<http://fimi.cs.helsinki.fi/data/>).

Hu, J., & Mojsilovic, A. (2007). High-utility pattern mining: A method for discovery of high-utility item sets. *Pattern Recognition, 40*(11), 3317–3324.

IBM Quest Data Mining Project, Quest synthetic data generation code. Available at (<http://www.almaden.ibm.com/cs/quest/syndata.html>).

Lan, G. C., Hong, T. P., & Tseng, Vincent S. (2011). Discovery of high utility itemsets from on-shelf time periods of products. *Expert Systems with Applications, 38*(5), 5851–5857.

Lin, C. W., Lan, G. C., & Hong, T. P. (2012). An incremental mining algorithm for high utility itemsets. *Expert Systems with Applications, 39*(8), 7173–7180.

Lee, C. H., Lin, C. R. & Chen, M. S. (2001). On mining general temporal association rules in a publication database. In *The 2001 IEEE international conference on data mining* (pp. 337–344).

Li, H. F., Huang, H. Y., & Lee, S. Y. (2011). Fast and memory efficient mining of high-utility itemsets from data streams: with and without negative item profits. *Knowledge and Information Systems, 28*(3), 495–522.

Li, Y., Ning, P., Wang, X. S., & Jajodia, S. (2003). Discovering calendar-based temporal association rules. *Data & Knowledge Engineering, 44*(2), 193–218.

Li, Y. C., Yeh, J. S., & Chang, C. C. (2008). Isolated items discarding strategy for discovering high utility itemsets. *Data & Knowledge Engineering, 64*(1), 198–217.

Liu, M., & Qu, J. (2012). Mining high utility itemsets without candidate generation. In *The 21st ACM international conference on information and knowledge management* (pp. 55–64).

Liu, Y., Liao, W., & Choudhary, A. (2005). A fast high utility itemsets mining algorithm. In *The utility-based data mining workshop* (pp. 90–99).

Ozden, B., Ramaswamy, S., & Silberschatz, A. (1998). Cyclic association rules. In *The 14th international conference on data engineering* (pp. 412–421).

Roddick, J. F., & Spiliopoilou, M. (2002). A survey of temporal knowledge discovery paradigms and methods. *IEEE Transactions on Knowledge and Data Engineering, 14*(4), 750–767.

Pei, J., Han, J., Mortazavi-Asl, B., Wang, J., Pinto, H., Chen, Q., et al. (2004). Mining sequential patterns by pattern-growth: The prefixspan approach. *IEEE Transactions on Knowledge and Data Engineering, 16*(11), 1424–1440.

Tseng, Vincent S., Chu, C. J., & Liang, T. (2006). Efficient mining of temporal high utility itemsets from data streams. In *The ACM KDD workshop on utility-based data mining* (pp. 1105–1117).

Yao, H., & Hamilton, H. J. (2006). Mining itemset utilities from transaction databases. *Data & Knowledge Engineering, 59*(3), 603–626.

Yeh, J. S., Chang, C. Y., & Wang, Y. T. (2008). Efficient algorithms for incremental utility mining. In *Conference on ubiquitous information management and communication* (pp. 229–234).

Yao, H., Hamilton, H. J., & Butz, C. J. (2004). A foundational approach to mining itemset utilities from databases. In *The 4th SIAM international conference on data mining* (pp. 482–486).